

Chapter 07

攝影機運鏡

(Cinemachine)

複習：上週重點

- [x] 製作了 Prefab (金幣、陷阱)。
- [x] 使用 Tag 區分物件。
- [x] 寫了 Interaction 腳本。

現在你的角色可以在很大的地圖裡跑來跑去。

但是！ 跑出畫面就看不到了。

本章目標

我們要幫遊戲請一位「專業攝影師」。

1. 認識 Cinemachine 套件。
2. 建立 Virtual Camera (虛擬攝影機)。
3. 設定 Follow (跟隨) 與 Dead Zone (緩衝區)。
4. 使用 Confiner (邊界限制) 防止穿幫。

傳統做法 vs 現代做法

以前 (Old School) :

- 自己寫 C# 腳本 `CameraFollow.cs` ◦
- 每一幀去抓玩家的座標 `transform.position` ◦
- 自己算平滑移動 `Vector3.Lerp` ◦
- 自己寫判斷不要超出邊界 `Mathf.Clamp` ◦
- 太累了！又容易有 Bug ◦

現在 (Modern) :

- Cinemachine : Unity 官方推出的智慧攝影機系統 ◦
- 不用寫一行程式碼，就能做到 3A 級的運鏡效果 ◦

安裝 Cinemachine

Cinemachine 是一個 Package，需要透過 Package Manager 安裝。
(註：Unity 2021 以後的 2D 模板通常預裝了，如果沒有請照做)

1. 上方選單 Window -> Package Manager。
2. 左上角切換為 Unity Registry。
3. 搜尋 **Cinemachine**。
4. 點選右下角 Install。

核心概念：大腦與眼睛

Cinemachine 的運作原理像這樣：

- **Unity Main Camera (Brain)**：真正負責輸出的攝影機。它身上會有一個 `CinemachineBrain` 元件。
- **Virtual Camera (Eye)**：虛擬攝影機，負責指揮 Brain 該去哪裡、看哪裡。
- **導演 (Director)**：Brain 會選擇「優先度最高」的 Virtual Camera 來顯示畫面。

我們只要控制 **Virtual Camera** 就好。

步驟 1：建立 2D 攝影機

1. 上方選單 Cinemachine -> Create 2D Camera ◦
 - (新版可能在 GameObject -> Cinemachine -> 2D Camera)
2. 你會發現 Hierarchy 多了一個物件 **CM vcam1** ◦
3. 原本的 Main Camera 旁邊多了一個紅色圖示 (它被 Brain 接管了) ◦

步驟 2：設定跟隨目標

選取 **CM vcam1**，看 Inspector：

- **Follow**：攝影機要跟著誰移動？ -> **將 Player 拖進來**。
- **Look At**：攝影機要旋轉對準誰？ -> **2D 遊戲通常留空** (因為我們不旋轉相機)。

Tip: 如果設定好之後畫面是歪的，檢查一下 Main Camera 的 Rotation 是否被動到了 (應該要是 0,0,0)。

步驟 3：調整鏡頭大小 (Lens)

覺得角色太小或太大？

- 找到 Lens 區塊。
- 調整 Orthographic Size (正交大小)。
 - 數值越小 = Zoom In (特寫)。
 - 數值越大 = Zoom Out (廣角)。
 - 一般手遊建議在 5 ~ 8 之間。

步驟 4：調整構圖 (Body)

Cinemachine 最強大的地方在於它的跟隨邏輯。
展開 **Body** (Transposer) 區塊：

- **Damping (阻尼)：**
 - 攝影機跟隨的「延遲感」。
 - 0 = 死死黏著玩家 (容易頭暈)。
 - 1 = 慢慢跟過來 (比較平滑自然)。
 - 建議 X, Y 設為 0.5 ~ 1 左右。

Dead Zone & Soft Zone

請看 Game 視窗裡的**藍色與紅色區域** (如果沒看到，請打開 Game 視窗的 Gizmos)。

- **Dead Zone (中間小方塊)：**
 - 玩家在這個區域移動時，**攝影機完全不動**。
 - 適合讓玩家做微小移動時畫面不要一直晃。
- **Soft Zone (藍色區域)：**
 - 玩家進入這裡時，攝影機開始慢慢移動追上。
- **Hard Limit (紅色邊界)：**
 - 玩家絕對出不去的邊界，攝影機強制移動。

Screen X / Screen Y

調整畫面構圖的中心點。

- 預設是 0.5, 0.5 (正中間)。
- 有些跑酷遊戲喜歡把主角放在左邊一點 ($\text{Screen X} = 0.3$)，讓右邊視野多一點，玩家比較好反應。

Lookahead (預判)

- Lookahead Time：攝影機預測玩家會去哪，提早移過去。
- Lookahead Smoothing：平滑度。
- 優點：跑得很快的遊戲很需要。
- 缺點：容易造成暈眩，**新手建議先設 0 (關閉)**。

邊界限制 (Confiner)

我們不希望玩家看到地圖外面的藍色虛空 (Blue Void)。
我們需要限制攝影機的移動範圍。

使用 Cinemachine Confiner 2D 擴充功能。

實作 Confiner：步驟 1

建立邊界範圍 (Bounding Shape)。

1. 在 Hierarchy 建立一個 Empty Object，命名 `CameraConfiner`。
2. 加入 Polygon Collider 2D。
3. **極度重要：勾選 Is Trigger！** (不然攝影機會把玩家撞飛)。
4. 點選 Edit Collider，拉動頂點，把你的「整個遊戲關卡」包起來。

實作 Confiner：步驟 2

設定 Virtual Camera ◦

1. 選取 `CM vcam1` ◦
2. 最下方 Extensions -> Add Extension -> Cinemachine Confiner 2D ◦
3. 將剛剛做好的 `CameraConfiner` 拖曳到 Bounding Shape 2D 欄位 ◦
4. Play !

現在攝影機碰到邊界就會停住，不會拍到外面了。

常見問題：Confiner 失效？

Q: 攝影機還是跑出去了？

A:

1. 檢查 Collider 是否有包住整個關卡。
2. 檢查 Confiner 裡的 Bounding Shape 有沒有 assignment。
3. 按一下 Confiner 裡的 **Invalidate Cache** 按鈕 (有時候它需要重新計算)。

進階技巧：螢幕震動 (Impulse)

爆炸、受傷時，螢幕震動能大幅增加打擊感。

1. 在 vCam 加入 Extension: **Cinemachine Impulse Listener**。
2. 在爆炸物或受傷腳本呼叫震動訊號 source。
 - (這部分需要程式配合，先知道有這功能即可)

多機位切換 (Cutsscenes)

如果你想做過場動畫，比如特寫魔王出場。

1. 建立第二個 vCam (**vCam2**)。
2. 把 **vCam2** 的 Follow 設為魔王。
3. **Priority (優先級)** :
 - vCam1 (Player): Priority = 10
 - vCam2 (Boss): Priority = 11
4. 因為 $11 > 10$ ，Brain 會自動平滑切換到 vCam2 ！

Pixel Perfect (像素完美)

如果你做的是 Pixel Art (像素風) 遊戲，會發現移動時畫面好像在閃爍 (Jitter) 或變形。

1. 在 Main Camera 加入 Pixel Perfect Camera 元件。
2. 設定你的 Reference Resolution (例如 320x180)。
3. 它會強制讓像素點對點，畫面會變得非常銳利清晰。

Cinemachine 幫我們解決了 90% 的攝影機問題。

1. **Virtual Camera**：控制跟隨與構圖。
2. **Damping**：讓運鏡更平滑。
3. **Dead Zone**：避免畫面過度晃動。
4. **Confiner**：限制視野不穿幫。

下週預告

攝影機動起來了，但背景看起來怪怪的？

因為背景跟著攝影機一起動，看起來像貼在鏡頭上一樣。

下週我們介紹 Parallax (視差滾動)：

- 遠景動得慢。
- 近景動得快。
- 創造偽 3D 的深度感。

Q & A

- 攝影機一直抖動？
 - 檢查 Update Method (Smart Update / Fixed Update)。
 - 如果是像素遊戲，試著用 Pixel Perfect。
- Confiner 沒反應？
 - 檢查 Collider 是否為 Polygon 或 Composite (Box Collider 有時不支援)。

(助教巡堂協助)